LIBRAIN: A Multi-Agent RAG System for Scientific Discovery

Engineering a Citation-Grounded Hypothesis Synthesis Framework in .NET 10

Eren Mutlu

Department of Computer Engineering, Karabük University, Türkiye

Correspondence: erennmutlu@outlook.com

Abstract

LIBRAIN is a research and engineering project that studies how the process of scientific discovery can be modeled and observed through a controlled multi-agent simulation. The system implements a Retrieval-Augmented Generation (RAG) pipeline that ingests open-access scientific papers, organizes them into a semantic index, synthesizes citation-grounded hypotheses from retrieved evidence, and submits each hypothesis to an automated multi-dimensional evaluation. The architecture consists of three reasoning agents, Reader, Synthesis, and Evaluator, supported by a cross-cutting audit-logging layer built on Microsoft Application Insights and structured correlation identifiers. The original four-agent design's Archivist role is deliberately collapsed into this logging layer, reflecting a separation between reasoning and observability concerns. A parallel Discovery Mode pipeline extrapolates beyond cited evidence: the portion of each hypothesis that is not directly supported by retrieved sources is explicitly tagged as a novelClaim, a structured field separating speculative inference from grounded synthesis, enabling cross-domain hypothesis generation while preserving citation discipline on the supported portion. The prototype is built on .NET 10 with the Anthropic.SDK NuGet package, Claude (Sonnet 4.6 for synthesis, Haiku 4.5 for evaluation), OpenAI text-embedding-3-small, and Qdrant (local Docker in development, Qdrant Cloud free tier in production). The complete source code, test suite, and architectural documentation are publicly available at github.com/erennmutlu1/librain, with end-to-end smoke runs returning 4/4 grounded citations at a quality score of 0.92 on in-corpus queries and a cross-run consistency study (N=5) characterising the stability of the Discovery Mode judging axes. A subsequent extension to 13 papers produced three cross-domain Discovery Mode demonstrations. Rather than focusing on automation, the project aims to illuminate how structured reasoning over scientific literature unfolds and how its trustworthiness can be measured.

Keywords: Artificial Intelligence, Scientific Discovery, Research Simulation, Multi-Agent Systems, Retrieval-Augmented Generation, LLM-as-a-Judge, .NET, Citation Grounding.

May 2026

Contents

1. INTRODUCTION

Discovery has always been at the heart of science. Every new idea grows out of the connections between existing ones, shaped by how we read, compare, and question what we already know. As research expands, the sheer volume of available information makes it harder for any single researcher to see the bigger picture or notice hidden links between distant subfields.

LIBRAIN was created to explore this space, not to replace the researcher but to study how discovery itself unfolds when its core motions are decomposed into observable, reproducible computational steps. The project asks whether the steps of scientific reasoning can be modeled in a digital setting: how knowledge is connected, how new hypotheses emerge, and how we might trace the otherwise invisible paths between ideas.

The current iteration of LIBRAIN moves beyond a purely conceptual framework. The current implementation is a working multi-agent system, written in .NET 10 and openly released at github.com/erennmutlu1/librain. The system ingests scientific papers from arXiv, organizes them in a semantic vector store, synthesizes hypotheses from retrieved evidence, and evaluates each hypothesis along multiple independent dimensions. Every step is recorded with a correlation identifier, so the path from question to answer is auditable through per-stage structured logs. By observing this process in a structured environment, LIBRAIN aims to help us better understand the rhythm of discovery and the patterns that guide it. It also does so on infrastructure familiar to enterprise software engineering, and not on the Python-centric ML monoculture.

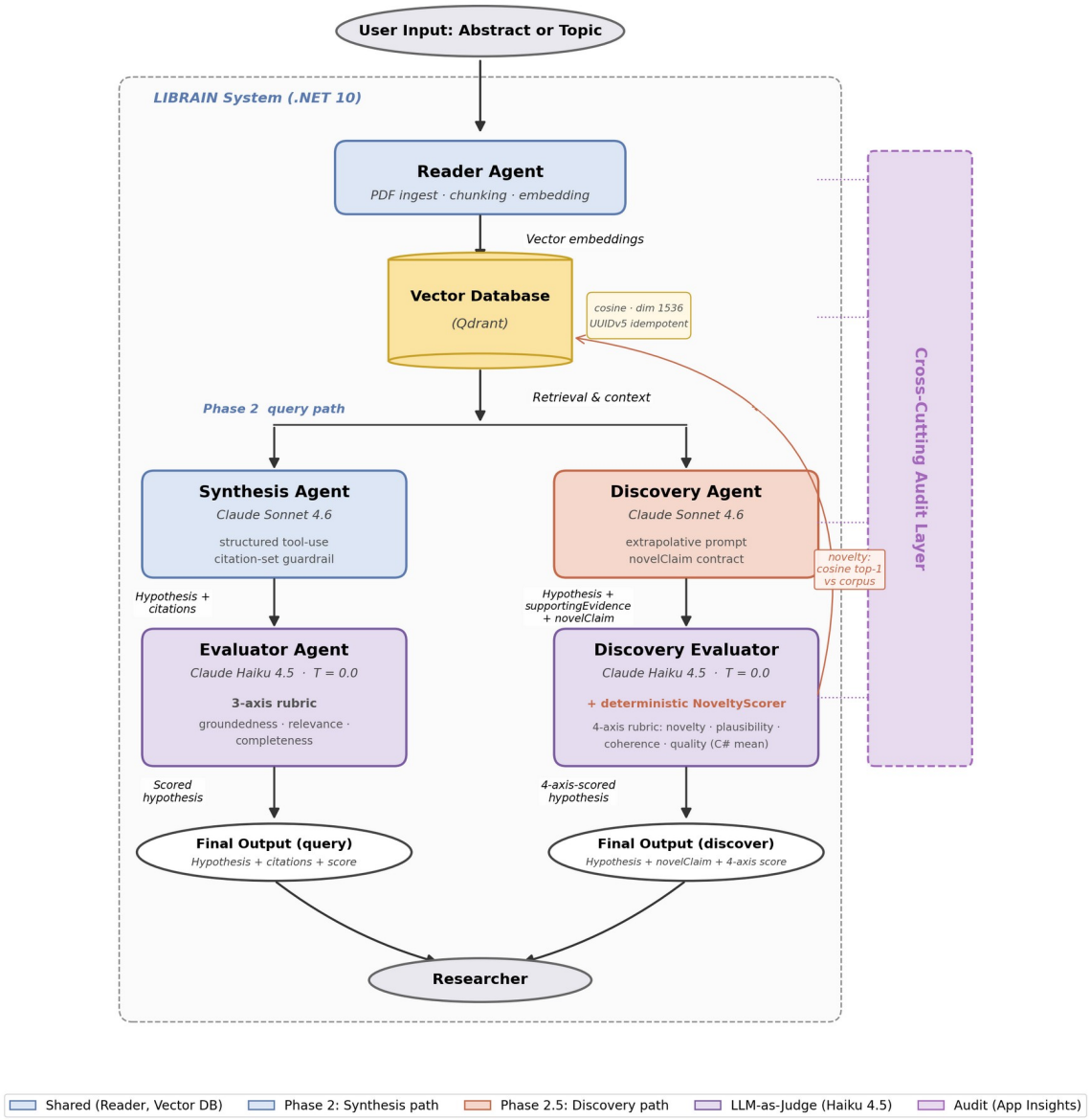

Figure 1. End-to-end LIBRAIN architecture showing both pipelines on a single shared infrastructure. The Reader Agent and Vector Database (Qdrant) are shared. The Phase 2 query path (POST /api/query) runs Synthesis Agent → Evaluator Agent on a 3-axis rubric. The Phase 2.5 discover path (POST /api/discover) runs Discovery Agent → Discovery Evaluator with a deterministic NoveltyScorer (cosine top-1 against the same Qdrant corpus) on a 4-axis rubric. The Cross-Cutting Audit Layer (Microsoft.Extensions.Logging scopes + Application Insights, keyed by correlation IDs) traces every stage of both pipelines. The Discovery validation loop is detailed in Figure 2b; empirical results are reported in Figure 7 and Figure 8.

2. METHODOLOGY

The implemented version of LIBRAIN is a focused engineering framework that models the essential steps of scientific reasoning through three modular agents and a cross-cutting audit layer. Each agent represents a critical, sequential stage in the discovery process, and the system is built on a Retrieval-Augmented

Generation (RAG) architecture for knowledge structuring. The goal is to observe how information is refined, connected, and critically assessed within a simulated research environment.

An earlier conceptual version of LIBRAIN proposed four agents (Reader, Hypothesis, Evaluator, Archivist). During implementation, two refinements were made. First, the Hypothesis Agent was renamed the Synthesis Agent, because its actual function is not free-form ideation but the synthesis of a citation-grounded answer from the retrieved evidence. Second, the Archivist Agent was reabsorbed into the runtime as a cross-cutting concern based on Application Insights and a correlation-identifier propagation pattern, not a standalone reasoning agent. These refinements reduce conceptual overlap and produce a leaner, more honest architecture.

2.1 Reader Agent

Triggered by a paper drop or an arXiv identifier, the Reader Agent is responsible for the entire ingestion pipeline. It extracts text from PDF sources using UglyToad.PdfPig (with a content-order extractor configured to preserve word spacing across multi-column scientific layouts), normalizes the text, and segments it into semantically coherent chunks of approximately 512 tokens with overlap. Each chunk is embedded using OpenAI's text-embedding-3-small model and persisted in a Qdrant vector collection along with structured metadata (paper identifier, title, authors, year, page number, section heading). Chunk identifiers are deterministic UUIDv5 hashes derived from a project namespace, ensuring that re-ingestion is idempotent. The resulting vector index serves as the immediately searchable and analyzable representation of the underlying corpus, eliminating the need for any separate extraction or organization step.

2.2 Synthesis Agent

The Synthesis Agent acts as the reasoning engine of the simulation. Given a user query, it embeds the query, performs a top-K vector search against the Qdrant collection, and submits the retrieved chunks together with the query to Anthropic Claude Sonnet 4.6 via a structured tool-use prompt. The model is instructed to produce a JSON response containing a hypothesis, a list of citation chunk identifiers, and a self-reported confidence score, and it is forbidden from citing identifiers outside the retrieved set. Citations that fail validation are dropped from the response, and the drop is recorded in the audit log so that prompt-engineering regressions can be detected from telemetry. This grounding loop is what allows the agent to perform a controlled "cognitive leap" (proposing connections between distant concepts) without drifting into fabrication.

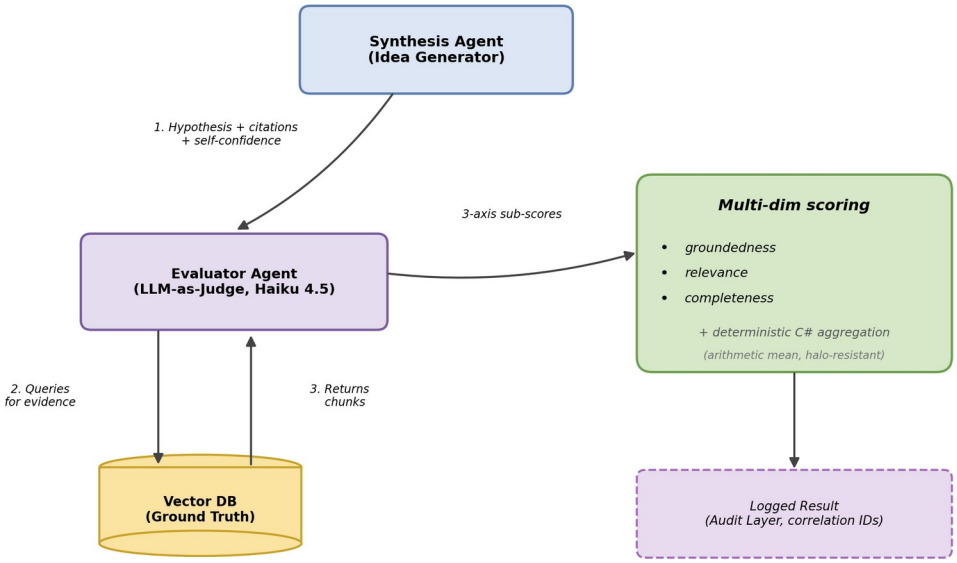

Figure 2. Phase 2 Synthesis–Evaluation verification loop. The Synthesis Agent proposes a hypothesis with citations. The Evaluator Agent independently re-queries the vector store, checks originality and plausibility on a multi-dimensional rubric (groundedness, relevance, completeness), and returns a structured score that is logged alongside the hypothesis. The Phase 2.5 Discovery validation loop, which adds deterministic novelty scoring and a four-axis rubric, is shown separately in Figure 2b.

2.3 Evaluator Agent

Acting as an impartial critic, the Evaluator Agent validates each hypothesis against the same evidence base used by the synthesis step. It runs as an LLM-as-a-Judge powered by Claude Haiku 4.5, configured with temperature 0.0 to maximize judging determinism. The output is a multi-dimensional score covering groundedness (does the hypothesis follow from the cited chunks?), relevance (does it address the user's actual query?), completeness (does it cover what the cited evidence supports?), and an aggregate quality score. The three sub-scores are returned by the language model independently, but the aggregation into a single quality value is computed deterministically in C# (arithmetic mean) rather than by the model itself, because language models tend to apply a halo effect when asked to combine sub-scores. The Evaluator does not see the Synthesis Agent's self-reported confidence, ensuring that the judgment is independent. Sub-scores that fall outside the [0, 1] interval are clamped, and clamping events are logged so that prompt-engineering regressions can be detected via telemetry.

2.4 Cross-Cutting Audit Logging

To ensure reproducibility and trust, LIBRAIN replaces the originally proposed Archivist Agent with a runtime audit-logging layer that pervades the entire pipeline. Every request is assigned a correlation identifier that flows through the Reader, Synthesis, and Evaluator stages via Microsoft.Extensions.Logging

scopes. When configured, Application Insights captures structured events from every stage (embedding generation, vector search, synthesis, and evaluation), recording per-stage chunk counts, token usage, elapsed times, citation-validation outcomes, and the per-dimension scores from the evaluation phase, all keyed by correlation identifier. This transparency converts the system from a black box into an observable laboratory of thought: any final hypothesis can be traced backwards through retrieval and reasoning to the chunks it depended upon.

2.5 Discovery Agent

Introduced in Phase 2.5 as a parallel pipeline to the Synthesis Agent, the Discovery Agent inverts the citation-grounded guardrail to enable extrapolation beyond the cited evidence. Given one or two topics together with an optional novelty target, it embeds each topic, performs independent top-K vector searches against the Qdrant collection, and submits the union of retrieved chunks to Anthropic Claude Sonnet 4.6 via a structured tool-use prompt. Unlike the Synthesis prompt, which forbids any claim not directly supported by retrieval, the Discovery prompt explicitly invites the model to propose a hypothesis that goes beyond the cited sources, with the unsupported portion returned as an explicit novelClaim field. The pipeline preserves citation discipline where it applies: claims tagged as supportingEvidence still validate against the retrieved set in C# and are dropped from the response if invalid, while novelClaim is exempt from that check by design, flagging the unsupported portion is the contract, not a defect. Discovery Mode is reachable through a separate POST /api/discover endpoint and does not replace the Synthesis → Evaluator path.

On the nature of novelClaim

The novelClaim field captures the model's inference beyond the retrieved evidence: by construction, it is the portion of the hypothesis not directly stated in any retrieved chunk. This makes it a hypothesis seed for investigation, not a validated scientific finding. The framing is intentional: citations support the structural framing of the hypothesis; the novelClaim is the speculative bridge. The four-axis evaluation (novelty, plausibility, structural coherence, quality) provides a structured assessment of how grounded versus speculative each output is. Readers reviewing Discovery Mode outputs should therefore treat novelClaims as suggestions for further investigation, analogous to a brainstormed hypothesis seed in human research rather than verified claims. This distinction is what allows the system to extrapolate productively without drifting into fabrication: the model is invited to speculate, but the speculation is fenced into a single labeled field that downstream consumers can route, weigh, or audit separately from the citation-backed scaffolding.

2.6 Discovery Evaluator and Novelty Scoring

Each Discovery hypothesis is scored on a four-axis rubric, two axes from a deterministic scorer and two from an LLM-as-a-Judge. The deterministic NoveltyScorer embeds the novelClaim text with the same OpenAI text-embedding-3-small model used for retrieval, performs a top-1 cosine search against the existing Qdrant corpus, and returns 1 minus the highest similarity as the novelty score. This makes novelty a measurement surface that depends only on embedding geometry, not on model preferences. The Discovery Evaluator Agent runs Claude Haiku 4.5 at temperature 0.0 with forced tool use to return two LLM-judged sub-scores, plausibility (does the novelClaim follow logically from the cited evidence even if

not stated directly?) and structural coherence (is the hypothesis a well-formed, testable scientific statement?), together with a free-text critique. The aggregate quality score is the arithmetic mean of all four axes, computed in C# rather than by the language model, mirroring the halo-resistance pattern of the Phase 2 Evaluator. As with Phase 2, every stage emits structured audit events keyed by correlation identifier so the per-axis decision and the underlying retrieval set can be reproduced from telemetry alone.

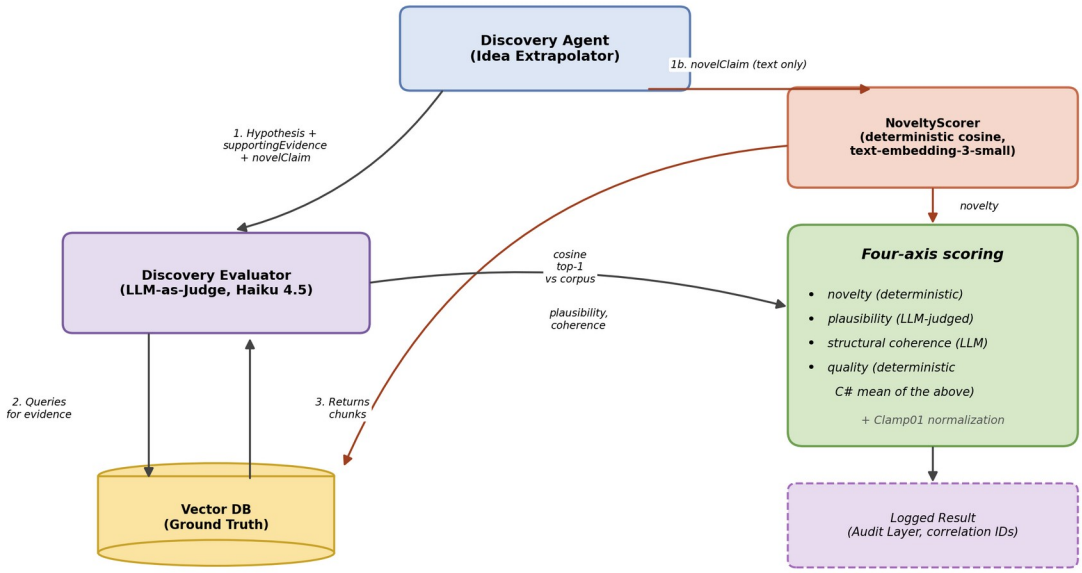

Figure 2b. Phase 2.5 Discovery validation loop. The Discovery Agent emits a hypothesis with a supportingEvidence array (validated against retrieval) and a novelClaim field (free text, exempt from citation validation by contract). The Discovery Evaluator queries the vector store for evidence and judges plausibility and structural coherence (LLM-as-Judge, Haiku 4.5, T=0.0). In parallel, the deterministic NoveltyScorer embeds the novelClaim and computes 1 – cosine similarity to the nearest existing chunk. The aggregate quality is the C# arithmetic mean of all four axes, halo-resistant by construction.

Component	Primary Role	Key Technology / Method
Reader Agent	Autonomous ingestion, chunking, embedding, semantic indexing	PdfPig, OpenAI text-embedding-3-small, Qdrant, deterministic UUIDv5 IDs
Synthesis Agent	Retrieval-grounded hypothesis generation with citation validation	Claude Sonnet 4.6 via structured tool-use, vector search, citation-set guardrail
Evaluator Agent	Independent multi-dimensional judging of each hypothesis	Claude Haiku 4.5 LLM-as-a-Judge (T=0.0), deterministic C# aggregation

Component	Primary Role	Key Technology / Method
Discovery Agent	Extrapolative hypothesis generation beyond cited evidence; flags unsupported portion as novelClaim	Claude Sonnet 4.6 via structured tool-use, selective citation validation (supportingEvidence only)
Discovery Evaluator	Four-axis evaluation of discovery hypotheses; combines deterministic novelty with LLM judging	Claude Haiku 4.5 (plausibility, coherence) + deterministic NoveltyScorer (cosine top-1) + C# aggregation
Audit Layer	End-to-end traceability and reproducibility	Microsoft.Extensions.Logging scopes, correlation IDs, Application Insights

Table 1. Summary of agent roles and core technologies in the implemented system.

3. ENGINEERING DECISIONS & CHALLENGES

This section outlines the technical requirements for each component and the concrete implementation challenges encountered during Phases 1 and 2. Several of these challenges produced deliberate, documented architectural decisions, not incidental fixes, and they are reported here in that spirit.

3.1 Reader Agent: Requirements and Challenges

Requirements. A vector store that supports both high-dimensional similarity search and per-chunk metadata filtering (paper, year, section), a deterministic ingestion pipeline so that re-runs do not duplicate points, and a chunking strategy that preserves enough context for grounding while staying within embedding-model limits.

Challenges encountered.

- **PDF text extraction on multi-column layouts.** Default PdfPig text extraction silently corrupted word boundaries on two-column scientific PDFs, which collapsed downstream section detection (0/15 sections recognized). Switching to PdfPig's ContentOrderTextExtractor restored 17 of 18 sections. This is now a documented learning anecdote: silent failures of this kind are why early manual validation matters more than passing unit tests.
- **Embedding rate limits.** OpenAI Tier 1 imposes a 40 K tokens-per-minute ceiling on text-embedding-3-small. The original 250 K-token batch budget triggered 429 responses. The pipeline now uses a 35 K-token batch budget with a 1.5-second inter-batch pace, which absorbs the limit without operator intervention.
- **Idempotent identifiers.** .NET 10 ships with Guid.CreateVersion7 but not v5. A small UUIDv5 (SHA-1) helper was added to derive deterministic point IDs from a project namespace and chunk key, ensuring that re-ingesting the same paper updates existing points instead of duplicating them in Qdrant.

3.2 Synthesis Agent: Requirements and Challenges

Requirements. A reasoning-capable LLM that can absorb several thousand tokens of retrieved context and produce structured output, with reliable mechanisms for forbidding citations to chunks that were not retrieved.

Challenges encountered.

- **Hallucination drift.** Free-form prompting produced confident-sounding answers with fabricated citations even when retrieval was good. The fix was to switch from prose output to a structured tool-use schema and to validate the cited identifiers against the retrieved set in C#, dropping any that fall outside it.
- **Prompt balance.** The system prompt must encourage synthesis ("connect ideas across the cited evidence") without inviting fabrication ("if the evidence is insufficient, say so"). Iterative refinement settled on an explicit unsupported_claims field that the model can populate when the evidence is thin, giving the system honest signal instead of rationalized output.

3.3 Evaluator Agent: Requirements and Challenges

Requirements. A judging model that is cheaper and faster than the synthesis model, configured for determinism, and a rubric that resists the halo effect typical of single-score LLM judging.

Challenges encountered.

- **Halo effect on aggregate scores.** When asked to produce a single overall score, the model tended to anchor on whichever sub-dimension was most salient in the prompt and let the others drift. The mitigation, drawn from the RAGAS and TruLens literature, is to elicit three independent sub-scores from the model (groundedness, relevance, completeness) along with three qualitative artifacts (a critique, an unsupported_claims list, and a missing_evidence list), and to aggregate the three numerical sub-scores deterministically in code.
- **Score range pollution.** Even with explicit instructions, the model occasionally returned values outside [0, 1] or omitted a field. A defensive Clamp01 helper normalizes the values, and any clamping event is logged at Information level so that prompt-engineering regressions can be detected from telemetry.

3.4 Stack Decision: From Cosmos DB to Qdrant Cloud

The original plan was to use Azure Cosmos DB (NoSQL, Vector Search) as the primary vector store, in part to align with the Microsoft AI-200 certification path. During Phase 1 development, two practical issues surfaced. First, the Cosmos emulator on macOS Apple Silicon is unstable, complicating local development. Second, paying for a Cosmos workload during the prototype phase yielded no learning advantage over a free local alternative. The pragmatic decision was to ship against Qdrant 1.17 in a local Docker container, with all data access confined to a single repository class so that any backend swap remained a one-file change. Phase 2.5 closed the loop: production hosting moved to the Qdrant Cloud free tier (AWS Frankfurt; 0.5 vCPU, 1 GB RAM, 4 GB disk), running the same engine, the same vector dimension, the same cosine distance metric, and the same UUIDv5 chunk identifiers as local development. The 218-chunk Phase 2 corpus uses well under one percent of the free-tier 250K-vector ceiling, the .NET configuration

auto-selects between local and cloud based on the presence of an API key in user-secrets, and there is no schema migration between dev and production, they are the same code path. Cosmos DB was retired as a production target post-Phase 2.5, since the local-versus-cloud parity removed the only remaining reason to migrate. This reflects a general principle of the project: choose the cheapest infrastructure that proves the architecture, and only pay for production once a paid alternative offers something the free one cannot.

3.5 Discovery Agent: Requirements and Challenges

Requirements. A discovery pipeline that runs in parallel with citation-grounded synthesis without compromising it; a contract that distinguishes supported claims from extrapolated ones rather than hiding the difference; a deterministic novelty signal that does not depend on the model's self-report; and an evaluation rubric that can score hypotheses whose value lies precisely in stepping outside the cited evidence.

Challenges encountered.

- **Premature abstraction between Synthesis and Discovery.** The two agents share surface shape (retrieve, prompt, return tool-use JSON, validate citations) but differ in intent: Synthesis is grounded by contract, Discovery extrapolates by contract. Extracting a shared base class was rejected because the divergence is in the system prompt and the citation guardrail policy, not in the orchestration shell. Keeping them as parallel concrete classes preserves clarity and avoids inheritance gymnastics that would obscure the most important contract difference.

- **Selective citation validation.** The naïve port of Phase 2's "every citation must exist in the retrieved set" rule would have dropped the entire novelClaim, which is the part of the response that is supposed to step beyond retrieval. The fix is to apply citation validation only to the supportingEvidence array, treat novelClaim as an unvalidated free-text field, and rely on the deterministic NoveltyScorer to quantify how far the novel part actually departs from the corpus. The validation guardrail still catches fabricated citations on the supported portion of the hypothesis; the novel portion is measured rather than gated.

- **noveltyTarget request knob.** An initial design exposed a noveltyTarget parameter (range [0.0, 1.0]) intended to let callers steer how aggressively the model extrapolated. A pre-ship validation protocol, three runs at noveltyTarget = 0.2 and three at noveltyTarget = 0.9 with retrieval held constant, measured a mean novelty differential of 0.0184 against a 2σ noise band of 0.0668, with the direction inverted. The knob did not control behavior in the intended way and was retired before reaching production users. The Discovery prompt now invites extrapolation unconditionally and the deterministic NoveltyScorer measures the result. The empirical detail is reported in Section 6.5.

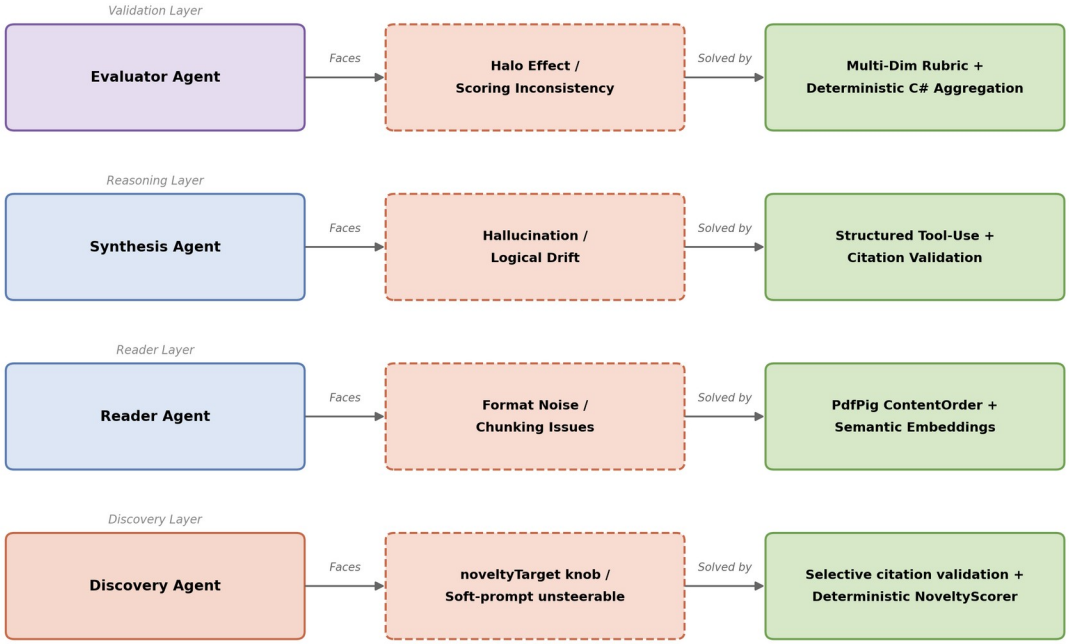

Figure 3. Component–challenge–solution map. Each agent layer is paired with the principal challenge encountered during implementation and the concrete mitigation strategy adopted.

Component	Specific Challenge	Mitigation Strategy
Reader Agent	Word-spacing corruption on two-column PDFs	Use PdfPig ContentOrderTextExtractor; manual validation gate before commit
Reader Agent	Embedding API rate limits (40 K TPM, Tier 1)	Lower batch token budget from 250K to 35K; insert 1.5 s inter-batch pacing
Synthesis Agent	Fabricated citations under free-form prompting	Structured tool-use schema; post-hoc citation-set validation in C#
Synthesis Agent	Tension between novelty and grounding	Explicit unsupported_claims field; iterative prompt refinement
Evaluator Agent	Halo effect on aggregate score	Multi-dimensional rubric; deterministic C# aggregation
Evaluator Agent	Out-of-range or missing sub-scores	Defensive Clamp01 normalization with telemetry-logged clamping events
Stack	Cosmos emulator instability on macOS ARM	Pivot to Qdrant for the MVP, with all data access confined to a single repository class; Cosmos retained as a future production-grade target

Component	Specific Challenge	Mitigation Strategy
Discovery Agent	Tension between hard-guardrail validation and soft-extrapolation prompt; naive port of citation validation would drop the entire novelClaim	Selective citation validation: supportingEvidence chunks validated against retrieval, novelClaim exempt by contract; deterministic NoveltyScorer measures novelty instead
noveltyTarget knob	Soft-prompt instruction did not measurably steer model behaviour; mean differential 0.0184 vs 2σ band 0.0668, with the direction inverted	Pre-ship validation gate retired the knob before reaching production users; deterministic NoveltyScorer measures the result instead

Table 2. Identified technical challenges and the corresponding mitigation strategies, drawn from the Phase 1 and Phase 2 implementation log.

4. DATASETS & TOOLS

This section details the physical resources and software architecture used in the LIBRAIN implementation, prioritizing function and reproducibility over unnecessary complexity.

4.1 Data Sources and Preparation

To precisely observe the system's reasoning capabilities without the noise of millions of documents, the prototype operates over a small, focused dataset. The Phase 2 evaluation corpus consists of five open-access papers from arXiv covering retrieval-augmented generation, hypothesis generation, and agentic AI for scientific discovery (arXiv:2005.11401, 2503.08979, 2504.05496, 2505.04651, 2508.14111). These papers were selected because they form a connected sub-graph of the literature. A small corpus is a feature, not a limitation here: it lets us verify that the system retrieves the canonically correct paper for a given query, instead of masking weaknesses behind volume.

4.2 Preparation Tools

Text extraction is handled by UglyToad.PdfPig (a managed .NET library) with the ContentOrderTextExtractor strategy. A C# segmentation service breaks each paper into approximately 512-token chunks with controlled overlap, attaching structured metadata (paper identifier, title, authors, year, section heading, page number) to each chunk before embedding. The choice of .NET over Python is deliberate: it serves as a counter-example to the assumption that modern LLM systems must be Python-first, and it aligns the project with the EU senior-backend ecosystem in which it is intended to function as a portfolio piece.

4.3 Knowledge Storage

Chunk vectors are stored in Qdrant 1.17 in both development and production. During development the system runs against a local Docker container with a persisted volume; production hosting uses the Qdrant Cloud free tier in AWS Frankfurt. The two environments are the same engine with the same vector dimension (1536), the same cosine distance metric, and the same deterministic UUIDv5 point identifiers,

so promotion from dev to production is a configuration change driven by the presence of an API key in user-secrets, not a migration. Each point uses a deterministic UUIDv5 identifier so that ingestion is idempotent and metadata is denormalized onto each point (Qdrant has no joins). A single repository class encapsulates all data access, so any future change of backend stays contained to that one file. Metadata fields (paper, year, section) are denormalized onto every Qdrant point so that filter-on-payload queries are available at the storage layer, with the query-API surface for them deferred to a later phase. The original plan to migrate production storage to Azure Cosmos DB Vector Search was retired post-Phase 2.5 once the local-versus-cloud parity removed the only practical reason to switch backends.

4.4 Core Reasoning Engine

The cognitive processing layer uses Anthropic Claude in two roles, accessed through the Anthropic.SDK NuGet package. Synthesis runs on Claude Sonnet 4.6, which carries the heavier reasoning burden. It is configured with a small positive temperature to permit the cognitive leap. Evaluation runs on the cheaper and faster Claude Haiku 4.5 at temperature 0.0 to maximize judging determinism. Embeddings are produced by OpenAI's text-embedding-3-small. All credentials are managed via .NET user-secrets in development. Production deployment will use Azure Key Vault. §5.5 documents a Phase B extension to 13 papers; this section describes the original Phase 1 / Phase 2 / Phase 2.5 seed corpus.

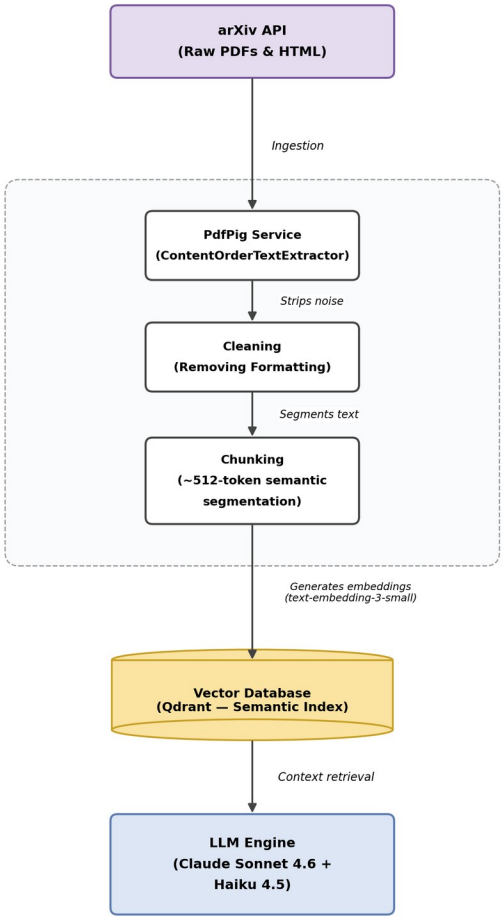

Figure 4. Data flow from raw arXiv source to the reasoning engine: ingest → noise stripping → semantic chunking → embedding → Qdrant vector index → context retrieval for the Synthesis Agent.

5. EXPERIMENTS

The objective of these experiments is not to benchmark the raw intelligence of any underlying language model, but to validate the process: to observe whether a structured multi-agent system can successfully simulate the mechanics of discovery in a controlled "scientific playground." Experiments 1 and 2 capture the behavior of the Reader and Synthesis Agents end-to-end, while Experiment 3 establishes the auditing characteristics of the Evaluator on a representative single-run query.

5.1 Experiment 1: Reading and Structuring

Goal. Verify that the Reader Agent autonomously ingests and semantically organizes raw research text, end-to-end.

Setup. The system processed five arXiv papers spanning RAG and agentic AI.

Active component. Reader Agent.

Result. All five papers ingested successfully, producing 218 chunks total in Qdrant. A canonical query ("How does retrieval-augmented generation mitigate hallucination?") returned a top-5 dominated entirely by chunks from arXiv:2005.11401 (Lewis et al., the founding RAG paper). The lazy collection-bootstrap path was exercised exactly once, as designed, and the previously identified ScrollAsync null-offset risk did not materialize. Observed retrieval latency held below 200 ms for top-5 queries against the local Qdrant instance.

5.2 Experiment 2: Synthesis and Citation Grounding

Goal. Assess whether the Synthesis Agent produces logically sound, citation-grounded hypotheses, and whether the citation-validation guardrail successfully blocks fabricated references.

Setup. The structured knowledge base from Experiment 1 was used as the context for a smoke query: "How does retrieval-augmented generation mitigate hallucination?"

Active component. Synthesis Agent.

Result. The agent returned a hypothesis with four citations, all of which validated against the retrieved set, and all four resolved to chunks from arXiv:2005.11401, the canonically correct source. The hypothesis used appropriate technical terminology (parametric vs. non-parametric memory) and accurately described the dual-memory architecture introduced by the cited paper. Self-reported synthesis confidence was 0.92. Synthesis latency was approximately 7 s, with a token usage of 6 365 in / 257 out, costing approximately 0.023 USD per synthesis call at current Sonnet 4.6 prices.

5.3 Experiment 3: Evaluation and Consistency

Goal. Test the stability and transparency of the auditing process: assess whether the Evaluator Agent applies its rubric consistently across runs, and whether the audit trail captures the full reasoning chain.

Setup. A representative query from Experiment 2 is submitted to the Evaluator. A control query ("What is the capital of France?") is used as a negative case to test the Evaluator's behavior on out-of-corpus topics: the original prediction was that relevance would drop, but the actual observation was that all three sub-scores held at 1.0 because the Synthesis Agent refused to fabricate and the Evaluator correctly rewarded that refusal (itself a positive design outcome). The scope of this experiment is the single-run characterization of the Evaluator's behavior. Cross-run variance studies are recognized as a complementary line of work.

Active components. Evaluator Agent and Audit Layer.

Observation. On the primary RAG-and-hallucination query, the Evaluator returned an aggregate quality score of 0.917, with sub-scores of 0.95 (groundedness), 0.95 (relevance), and 0.85 (completeness). The Evaluator deducted 0.15 from completeness for an actual gap in the cited evidence (the cited chunks describe RAG's architecture but do not detail the cognitive mechanism by which retrieval suppresses hallucination), rather than rubber-stamping the hypothesis. This is the form of judging behavior the rubric was designed to elicit. Application Insights captured stage-by-stage timings, token usage, citation-validation outcomes, and per-dimension scores under a single correlation identifier, supporting the audit-trail completeness criterion. End-to-end query latency, including evaluation, was 13.5 s. Cost per query was approximately 0.030 USD.

Experiment	Focus Area	Active Components	Headline Metric
Exp 1: Reading	Data ingestion & semantic structure	Reader Agent	5 papers / 218 chunks; observed retrieval < 200 ms
Exp 2: Synthesis	Citation grounding & logical synthesis	Synthesis Agent	4/4 citations valid, confidence 0.92
Exp 3: Evaluation	Judging reliability & audit trail	Evaluator Agent + Audit Layer	Quality 0.917 on primary query; per-stage timings, scores, and counts captured in the audit trail
Exp 4: Discovery	Cross-domain extrapolation & 4-axis sensitivity profiles	Discovery Agent, NoveltyScorer, Discovery Evaluator, Audit Layer	NoveltyScorer Δ = 0.2577 (5× noise floor); plausibility classified GENUINE SIGNAL (N = 5); noveltyTarget knob retired pre-ship
Exp 5: Extended Corpus	Cross-domain generalization & portfolio demos	Reader Agent (ingest), Discovery Agent, Discovery Evaluator, Audit Layer	10/10 valid runs; cherry-picked 3 cross-domain demos at quality 0.66, 0.67, 0.59

Table 3. Summary of the experimental matrix (Phase 2 and Phase 2.5) and headline observations.

5.4 Experiment 4: Discovery Mode Validation (Phase 2.5)

Goal. Validate the Discovery pipeline end-to-end: confirm that NoveltyScorer produces a meaningful in-corpus / off-corpus differential, that the noveltyTarget request knob behaves as designed before exposing it to callers, and that the multi-axis evaluation rubric distinguishes signal from noise across repeated runs.

Setup. All Discovery experiments ran against the same 218-chunk Qdrant Cloud corpus used in Experiments 1–3. Two locked topic pairs were used throughout: an in-corpus pair ("retrieval-augmented generation" / "hypothesis evaluation") that probes both topics inside the corpus, and an off-corpus pair ("retrieval-augmented generation" / "protein folding dynamics") that places one topic deliberately outside the corpus to elicit a cross-domain extrapolation.

Active components. Discovery Agent, NoveltyScorer, Discovery Evaluator Agent, and Audit Layer.

Result. Four sub-experiments were run. (a) NoveltyScorer baseline: in-corpus topic string scored 0.3861, off-corpus topic string scored 0.6438, a differential of 0.2577, five times the empirically measured noise floor of 0.05, confirming that cosine-distance novelty discriminates corpus membership cleanly at the embedding layer. (b) noveltyTarget validation gate: three runs at noveltyTarget = 0.2 (mean 0.3873, std 0.0331) versus three runs at noveltyTarget = 0.9 (mean 0.3689, std 0.0334), with retrieval held identical across all six. The mean differential of 0.0184 fell below the 2σ threshold of 0.0668 (ratio 0.276), and the direction inverted, the knob failed and was retired. (c) End-to-end smoke pairs: in-corpus quality 0.4997 (novelty 0.3992), off-corpus quality 0.5279 (novelty 0.4838); the 0.0846 end-to-end novelty differential preserves about one third of the standalone signal, with the compression mechanism analyzed in Section 6.5. (d) Cross-run consistency study, N = 5 per pair: plausibility classified as GENUINE SIGNAL (in-corpus 0.498 ± 0.143; off-corpus 0.612 ± 0.115), structural coherence classified as AMBIGUOUS-as-floor (in-corpus 0.702 ± 0.020; off-corpus 0.728 ± 0.038), and novelty stable within 0.03 standard deviation. Total Phase 2.5 experimental cost was approximately 0.74 USD across 23 LLM-judged runs. Full per-run telemetry is captured in the audit log under Phase 2.5 correlation identifiers.

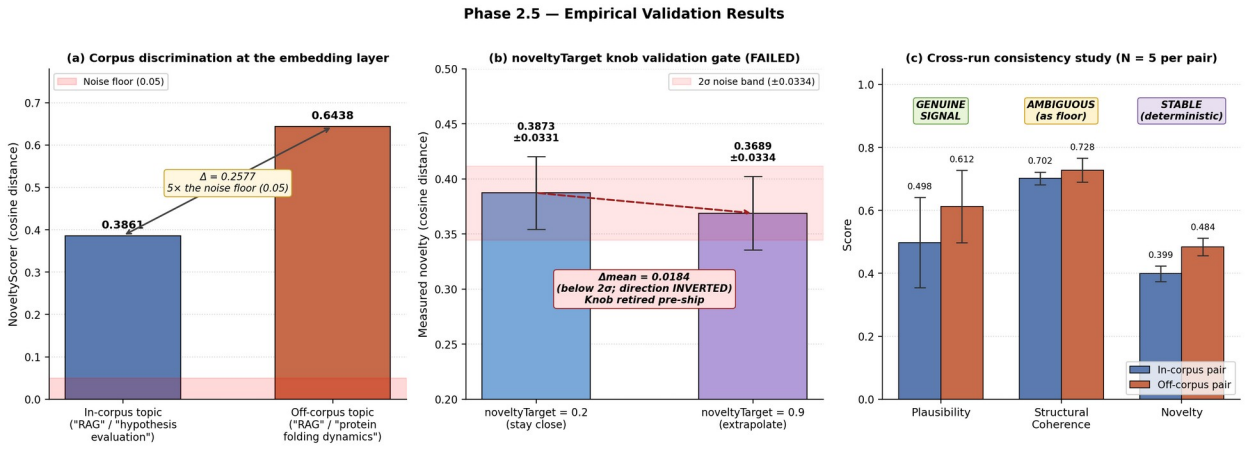

Figure 7. Phase 2.5 empirical validation results. (a) Standalone NoveltyScorer cleanly discriminates corpus membership at the embedding layer (Δ = 0.2577, five times the noise floor). (b) The noveltyTarget knob failed its pre-ship validation gate: the mean differential between target = 0.2 and target = 0.9 was 0.0184, well below the 2σ band of 0.0668, and the direction inverted, the knob was retired before reaching production users. (c) Cross-run consistency study (N = 5 per locked pair) classifies plausibility as a GENUINE SIGNAL axis, structural coherence as AMBIGUOUS-as-floor, and novelty as STABLE.

5.5 Experiment 5: Extended Corpus Demonstrations (Phase B)

Goal. Validate Discovery Mode's cross-domain capability beyond the original 5-paper RAG / agentic-AI corpus, and produce concrete demonstrations for portfolio dissemination.

Setup. The Phase 2 / 2.5 corpus was extended from 5 to 13 papers, adding eight open-access arXiv works spanning drug discovery (ChemCrow, arXiv:2304.05376; Foundation Model in Biomedicine, arXiv:2503.02104; PharmAgents, arXiv:2503.22164), climate and weather forecasting (GraphCast, arXiv:2212.12794; Pangu-Weather, arXiv:2211.02556; Aurora, arXiv:2405.13063), and computational neuroscience (Centaur, arXiv:2410.20268; Large Language Models for EEG, arXiv:2506.06353). The expanded corpus contains 610 chunks across all 13 papers (Phase 1 seed: 5 papers / 218 chunks; Phase B: 8 papers / 392 chunks). Papers were selected by recognition (well-known systems papers and major recent surveys) and by maintaining at least one keyword bridge to the existing RAG / agentic-AI corpus to support retrieval.

Active components. Reader Agent (for ingestion of new papers), Discovery Agent, NoveltyScorer, Discovery Evaluator Agent, and Audit Layer.

Protocol. Ten topic pairs were defined a priori before any results were inspected, distributed as: three drug-discovery pairs, three climate / energy pairs, two neuroscience–learning pairs, and two pure cross-domain pairs (where one topic explicitly comes from a distant domain). All ten were executed once sequentially via POST /api/discover with topK=5. No retries, no parameter sweeps. Cherry-picking was applied only after all ten outputs were collected, using a fixed scoring formula combining a manual legibility score (1–5), a manual surprise score (1–5), and a quality-score floor of 0.6, to select three demonstrations for portfolio publication.

Result. Ten of ten runs returned HTTP 200 with zero citation-contract violations; all supportingEvidence entries resolved to the retrieved chunk set as specified in §2.5. Quality scores ranged from 0.43 to 0.67; plausibility (the LLM-judged content-discriminating axis identified in §6.5) ranged from 0.28 to 0.72. The three cherry-picked runs were: (i) “retrieval-augmented generation” × “de novo molecular design” (quality 0.66, plausibility 0.68); (ii) “weather foundation models” × “renewable energy planning” (quality 0.67, plausibility 0.72); (iii) “drug discovery” × “climate adaptation” (quality 0.59, plausibility 0.62). Per-pair detailed scoring is reported in Table 5.

#	TopicA × TopicB	Quality	Plausibility	Novelty	Coherence
1	large language models × drug-target interaction prediction	0.4294	0.28	0.3883	0.62
2	**retrieval-augmented generation × de novo molecular design**	**0.6628**	**0.68**	**0.4884**	**0.82**
3	agentic AI × clinical trial design	0.5196	0.42	0.4588	0.68
4	deep learning weather forecasting × agricultural yield prediction	0.5467	0.42	0.5401	0.68
5	climate adaptation × language models	0.6324	0.72	0.3971	0.78
6	**weather foundation models × renewable energy planning**	**0.6703**	**0.72**	**0.5110**	**0.78**
7	large language models × human learning curriculum	0.5216	0.42	0.4647	0.68
8	brain inspired architectures × hypothesis generation	0.4823	0.42	0.3469	0.68
9	protein folding × weather forecasting	0.4569	0.28	0.4706	0.62

#	TopicA × TopicB	Quality	Plausibility	Novelty	Coherence
10	drug discovery × climate adaptation	0.5942	0.62	0.4125	0.75

Table 5. Phase B extended corpus demonstration results across 10 topic pairs. Highlighted rows indicate the three cherry-picked for portfolio publication. All ten pairs executed once sequentially against the 13-paper corpus.

Cost. End-to-end Phase B experimental cost was approximately 0.55 USD, comprising approximately 0.10 USD for 8-paper ingestion (OpenAI text-embedding-3-small) and approximately 0.45 USD for ten Discovery runs (Claude Sonnet 4.6 synthesis + Claude Haiku 4.5 evaluation per call). Discussion of the three cherry-picked outputs and what they reveal about cross-domain generalization is reported in Section 6.6.

6. RESULTS & DISCUSSION

This section reports the behavior observed during smoke testing and discusses what those results imply about the design. Anticipated outcomes from the original proposal are matched against measured results.

6.1 Quality of Semantic Organization

The Reader Agent successfully clusters documents by conceptual similarity, not keyword overlap. The success indicator from the original proposal was that a query for retrieval-augmented generation should surface the canonical Lewis et al. paper, even when its title does not appear in the query. This was met: the top five chunks for that query came entirely from arXiv:2005.11401. The originally feared failure mode (chunks too small to preserve semantic context) was avoided by the 512-token chunking strategy. Section detection, which is downstream of chunk quality, recovered from 0/15 to 17/18 sections after the PdfPig extraction strategy was corrected, validating the broader observation that ingestion-layer quality dominates retrieval-layer quality.

6.2 Hypothesis Synthesis: The Goldilocks Zone

The most critical metric for the Synthesis Agent is the trade-off between creativity and plausibility. The smoke results land near the target zone: high plausibility (groundedness 0.95, relevance 0.95) with moderate-to-high informational density (completeness 0.85). The Evaluator's 0.15 completeness deduction is methodologically encouraging, since it demonstrates that the judge does not rubber-stamp confident-sounding output but instead identifies real gaps in the cited evidence. The originally anticipated rejection rate of 30–40% under high-temperature settings has not been measured systematically, because the citation-set validation drops fabricated indices before they reach the Evaluator, masking the upstream fabrication rate. This is recognized as a complementary line of measurement.

6.3 System Limitations

Latency. End-to-end query latency is approximately 13.5 s (synthesis ≈7 s plus evaluation ≈6 s, executed sequentially). This exceeds the original sub-5-second target and reflects the cost of running two LLM calls per query. Prompt caching and response streaming are standard mitigations applicable here, and parallelization of the synthesis–evaluation pair is a natural further optimization.

Context window. While RAG mitigates the issue, the synthesis model still cannot "see" the entire library at once and is bottlenecked by retrieval relevance. Cross-domain queries that require chaining evidence across two distant papers have not yet been stress-tested.

Bias propagation. If the input corpus is biased toward a particular position, the system will reinforce that position instead of challenging it. The current corpus is small and topically homogeneous, so this limitation is acute, and any corpus expansion benefits from deliberate viewpoint heterogeneity to test bias propagation.

Cost. At approximately 0.030 USD per query, sustained interactive use is feasible at the prototype scale but would need re-engineering (prompt caching, smaller judging models, batched evaluation) at production scale.

6.4 Implications for Future Research and Graduate Study

Beyond its immediate technical contributions, LIBRAIN is designed as a foundation for future research-oriented graduate study. The framework already exposes several open research directions, including scalable hypothesis evaluation across larger corpora, long-term reasoning consistency under repeated re-runs, reliable cross-domain semantic retrieval, and the design of judging models that resist the halo effect without sacrificing aggregate interpretability. These aspects align naturally with the objectives of a research-oriented MSc or future PhD programme in artificial intelligence and computational science. The modular, transparent, and reproducible architecture of LIBRAIN, with a public GitHub repository, deterministic identifiers, structured audit trails, and a clean separation between reasoning and infrastructure, also makes it well suited for extension within an academic research environment. The system was developed as an independent research initiative outside formal graduate training and is intended to be further explored and refined through research-assistantship roles or formal graduate research programmes.

Dimension	Score (0-1)	Description of Criteria
Groundedness	0.0 (Fail)	The hypothesis contradicts the cited evidence or relies on uncited claims.
	1.0 (Pass)	Every load-bearing claim is supported by a cited chunk in the retrieved set.
Relevance	0.0 (Fail)	The hypothesis does not address the user query.
	1.0 (Pass)	The hypothesis answers the user query directly and on-topic.
Completeness	0.0 (Fail)	Major aspects of the question are unaddressed despite available evidence.
	1.0 (Pass)	The hypothesis covers the available evidence comprehensively.
Quality (aggregate)	Computed	Deterministic C# aggregation of the three sub-scores; not produced by the language model.

Table 4. Multi-dimensional rubric used by the Evaluator Agent. Sub-scores are produced independently by the judging LLM; the aggregate quality score is computed in code to resist halo effects.

6.5 Discovery Mode: Validation and Findings

The Phase 2 pipeline (POST /api/query) answers questions from retrieved evidence and is, by design, conservative; the Synthesis Agent's system prompt forbids any claim not directly supported by a cited source. Phase 2.5 added a parallel POST /api/discover pipeline that inverts that guardrail: given one or two topics, the system proposes a hypothesis that extrapolates beyond the cited sources and explicitly flags the unsupported portion as novelClaim. The pipeline is evaluated on a four-axis rubric: deterministic novelty (cosine distance to the nearest existing chunk), LLM-judged plausibility, LLM-judged structural coherence, and a deterministic aggregate quality score. Three empirical findings shape the system as it ships.

The first finding is that noveltyTarget, a request parameter that mapped into the system prompt as a soft calibration instruction on a [0.0 = stay close, 1.0 = extrapolate] scale, did not measurably steer model behavior. A validation protocol ran three calls at noveltyTarget = 0.2 and three at noveltyTarget = 0.9 against an in-corpus topic pair, with retrieval held identical across all six runs. The mean novelty differential was 0.0184 against a 2σ noise band of 0.0668, below the gate threshold and, more strikingly, with the direction inverted: claims at noveltyTarget = 0.9 were measurably *less* novel by cosine distance than at 0.2. The mechanism is that high-target prompts elicited domain-vocabulary-rich hypotheses landing nearer the corpus center in embedding space, while low-target prompts produced peripheral phrasing that drifted further. The knob was dropped post-empirically. The narrower implication is that cosine-distance novelty, as implemented here, is a *measurement* surface for what the model produced, not a *control* surface that prompt instructions can steer. The validation gate worked as designed: a parameter that did not survive its empirical test was retired before reaching production users. The same domain-vocabulary mechanism explains the end-to-end novelty differential: the 0.2577 separation between in-corpus and off-corpus topic strings (measured at the standalone NoveltyScorer) compresses to 0.0846 once the LLM produces hypothesis text, a 33% preservation. The signal direction is correct, but its magnitude is below the 0.10 acceptance target. A cleaner future-work direction is using a different embedding model for novelty than for retrieval, so the novelty space is not pre-coupled to the corpus.

The second finding came from a cross-run consistency study (N = 5 per locked smoke pair) conducted to disambiguate an early flag, both pairs had returned identical 0.42 / 0.68 plausibility / coherence values on the single-shot ship smoke. The classification gate (std < 0.01 → ANCHOR, the model is not actually evaluating; 0.01 ≤ std ≤ 0.05 → AMBIGUOUS; std > 0.05 → GENUINE SIGNAL) placed plausibility firmly in GENUINE SIGNAL territory on both pairs (in-corpus 0.498 ± 0.143; off-corpus 0.612 ± 0.115). The single-shot collision was therefore coincidental, not structural. Structural coherence landed AMBIGUOUS on both pairs (in-corpus 0.702 ± 0.020; off-corpus 0.728 ± 0.038), with three to four distinct values per pair clustered in a narrow band around 0.70. The reading is that structural coherence functions as a well-formedness floor: once a hypothesis meets a baseline of multi-sentence-with-supporting-evidence form, the rubric does not finely discriminate further. Novelty was stable as expected, with within-pair standard deviation under 0.03, the deterministic measure does not depend on novelClaim paraphrase variation. A surprise emerged from the same study and is worth foregrounding even on a two-pair sample: off-corpus plausibility (0.612) was materially higher than in-corpus plausibility (0.498). The interpretation we currently favor is that cross-domain prompts force the LLM to articulate the inferential bridge explicitly because the analogy is not shared context, while in-domain prompts permit gloss with assumed shared

vocabulary; the evaluator notices the missing scaffolding in the latter. This inverts what a naive novelty/plausibility tradeoff would predict and deserves a follow-up study with broader corpus diversity.

The third finding, drawn together from the cross-run study, is the closing thesis: the four rubric axes have measurably different *sensitivity profiles*, not equivalent measures of hypothesis quality. Novelty is a deterministic measurement surface, decoupled from the LLM-judged axes and stable across paraphrases. Plausibility is the content-discriminating LLM-judged signal, it varies with the inferential structure of the hypothesis. Structural coherence is a well-formedness gate that bounds rather than ranks; most well-written hypotheses cluster in a narrow band around 0.70. The aggregate quality score is a useful proxy that flattens these distinct profiles. The practical implication is that any production deployment of LIBRAIN should report all four axes rather than collapse to a single quality number; single-score reporting hides exactly the information that makes the multi-axis rubric worth running. This is the lesson the empirical work earned, and it shapes both the system as it ships and the open questions the next phase inherits.

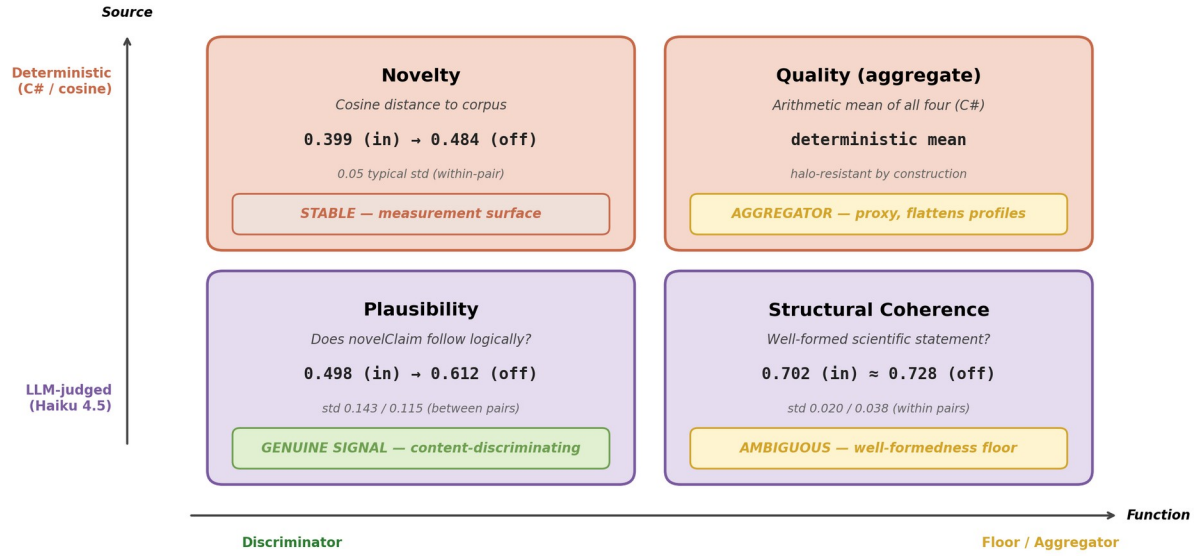

Figure 8. Four-axis sensitivity profiles. The two deterministic axes (Novelty, Quality) play distinct roles, one is a measurement surface, the other an aggregator. The two LLM-judged axes diverge sharply: plausibility discriminates content (GENUINE SIGNAL, std 0.143/0.115 between pairs) while structural coherence functions as a well-formedness floor (std 0.020/0.038 within pairs). The single aggregate quality score flattens these distinct profiles, reporting all four axes preserves the information the multi-axis rubric was designed to surface.

6.6 Cross-Domain Generalization at Broader Corpus Scale

The Phase B extended demonstrations probe whether Discovery Mode's cross-domain capability, formally validated on the in-corpus / off-corpus pair in §6.5, generalizes when the corpus itself spans heterogeneous scientific domains. The 13-paper extended corpus deliberately combines the original RAG / agentic-AI seed with three distant domains (drug discovery, climate and weather forecasting, computational neuroscience), and ten topic pairs were run against it before any cherry-picking.

The first observation is that the citation-validation contract held cleanly across all ten runs: every supportingEvidence entry resolved to a retrieved chunk, and the novelClaim field was non-empty in every

response. The system never fabricated a citation under the broader corpus, including on pairs where one or both topics were intentionally outside the seed corpus's RAG / agentic-AI focus. This replicates at corpus scale the citation discipline established in §6.2 and §6.5.

The second observation concerns the cherry-picked demonstrations. The three selected runs, namely "RAG × de novo molecular design", "weather foundation models × renewable energy planning", and "drug discovery × climate adaptation", represent qualitatively different bridge types. The first transfers a methodological idea (RAG's retrieval-grounded generation) across modalities (text-to-molecule). The second proposes a downstream application (weather forecasting models repurposed as probabilistic scenario engines) for a system originally built for a related but distinct purpose. The third closes a longer cross-domain loop (climate signals informing drug discovery target prioritization). All three were generated by the same Discovery Agent prompt, against the same corpus, with no per-pair tuning; the bridges were produced by retrieval distribution and the model's inference, not by hand engineering.

The third observation involves the framing question that each cherry-picked novelClaim implies. Each can be summarized as a single question that frames the hypothesis seed for a reader: "What if RAG's hallucination-reduction trick also stopped chemically implausible molecules from being generated?", "What if global weather AI could double as a probabilistic scenario engine for sizing renewable infrastructure?", and "What if drug discovery agents could read climate forecasts to anticipate emerging disease patterns?". Each question is legible to a non-specialist within a few seconds while remaining recognizable as a non-trivial speculative leap to a specialist in the relevant field, which is the legibility target for portfolio dissemination.

The fourth observation is a limitation. The 4-axis scores of the cherry-picked runs fall in the moderate range (quality 0.59–0.67, plausibility 0.62–0.72). This is consistent with the §6.5 finding that plausibility is the discriminating content-axis: pairs with higher plausibility produce stronger demonstrations, and pairs with low plausibility (Runs 1 and 9 at 0.28) produced less defensible novelClaims even when their narrative ambition was high. The Phase B set thus also confirms that the four-axis rubric does discriminate output quality at corpus scale, not just on the original two locked pairs.

The fifth observation is the role of the corpus itself. Two cherry-picked novelClaims (Demos B and C) explicitly name papers from the Phase B additions (Aurora, GraphCast, PharmAgents), confirming that the retrieval layer was successfully bringing the new domain content into Discovery Agent context. The corpus expansion is thus not decorative: the cross-domain bridges are anchored in retrieved chunks from the new papers, and the citation validation contract enforces that anchoring.

These observations together support a stronger claim than v12 could make at the 5-paper corpus scale: Discovery Mode produces legible, citation-anchored cross-domain hypothesis seeds across heterogeneous scientific domains without per-pair tuning, with a four-axis rubric that meaningfully discriminates output quality. The system's value as research scaffolding is correspondingly broader than the original RAG / agentic-AI focus might have suggested.

7. PROJECT TASKS & IMPLEMENTATION PLAN

This section describes the five phases of the LIBRAIN implementation, including the Phase 2.5 Discovery Mode extension that was scoped after Phase 2 shipped. The phased structure reflects a deliberate

decision to ship a working ingestion-and-retrieval pipeline before layering in synthesis and evaluation, to extend the validated pipeline with a parallel extrapolative branch in 2.5, and to defer production-grade infrastructure until the architectural design has been validated. The Cosmos-to-Qdrant pivot, in particular, removed emulator-related friction during local development by replacing a paid managed service with a free, locally-runnable alternative, while keeping all data access in a single repository class so the swap remained a one-file change.

7.1 Phase 1: Data Collection & Reader Agent

- Curated initial corpus of 5 open-access arXiv papers and validated metadata extraction.
- Built the .NET 10 ingestion pipeline (PdfPig text extraction, semantic chunking, OpenAI embeddings).
- Implemented Qdrant repository with deterministic UUIDv5 identifiers and lazy-bootstrap collection provisioning.
- Shipped POST /api/papers/ingest and GET /api/papers endpoints; wired Application Insights end-to-end.
- All 5 papers ingested successfully, producing 218 chunks; chunker test suite green at 7/7.

7.2 Phase 2: Synthesis, Evaluation & Query Pipeline

- Implemented the Synthesis Agent with structured tool-use prompting and citation-set validation that drops invalid citations with audit-log telemetry.
- Implemented the Evaluator Agent as an LLM-as-a-Judge with multi-dimensional rubric and deterministic C# aggregation.
- Shipped POST /api/query with the embed → search → synthesize → evaluate pipeline behind a single correlation identifier.
- Smoke test green: quality 0.917, all citations valid, per-stage timings/scores/counts captured in the audit trail.
- Test suite expanded to 19/19 passing across chunker, citation, and scoring components (subsequently grown to 30/30 with the Phase 2.5 additions).

7.3 Phase 2.5: Discovery Mode (Extension)

- Implemented the Discovery Agent as a parallel pipeline to the Synthesis Agent, with an inverted system prompt that explicitly invites extrapolation beyond the cited evidence and returns the unsupported portion as a flagged novelClaim field.
- Added a deterministic NoveltyScorer (cosine distance to the nearest existing chunk) and a Discovery Evaluator Agent (Haiku 4.5, plausibility and structural coherence axes), with the four-axis aggregate computed in C# to preserve halo-resistance.
- Shipped POST /api/discover with selective citation validation: supportingEvidence chunks must validate against the retrieved set, novelClaim is exempt by contract.

- Migrated production storage from local Docker Qdrant to the Qdrant Cloud free tier (AWS Frankfurt) with no schema migration; retired the Azure Cosmos DB target after the local-versus-cloud parity removed the only remaining reason to switch backends.
- Ran a pre-ship validation protocol on the noveltyTarget request knob (three runs at 0.2 versus three at 0.9). The knob failed to steer behavior in the intended direction (|Δmean| 0.0184 versus 2σ 0.0668, direction inverted) and was retired before reaching production users.
- Conducted a cross-run consistency study (N = 5 per locked smoke pair) classifying plausibility as a GENUINE SIGNAL axis and structural coherence as an AMBIGUOUS-as-floor axis, surfacing the sensitivity-profiles thesis reported in Section 6.5.
- Test suite grown from 19 to 30 passing tests with NoveltyScorer and DiscoveryScoring unit tests added.

7.4 Phase 3: Polish, Performance & Showcase

7.4.A Phase 3.A: Showcase

- Corpus expansion beyond the focused 5-paper Phase 2 set to 13 papers spanning drug discovery, climate forecasting, and computational neuroscience, exercising cross-domain Discovery Mode at broader corpus scale.
- Static demonstration set on the portfolio site (erenmutlu.me/projects/librain) featuring three cherry-picked Discovery Mode outputs with citation-resolved supporting evidence, 4-axis scores, and reader-facing framing questions. Substitutes for the originally planned interactive frontend by providing a deterministic, always-available, zero-cost-per-view demonstration that nonetheless renders real API outputs.

7.4.B Phase 3.B: Deployment & Performance (deferred, optional)

- Minimal Next.js or Blazor frontend on Azure Static Web Apps for interactive demonstration.
- API deployment on Azure Container Apps with the existing Qdrant Cloud production index.
- Anthropic prompt caching to reduce per-query token cost and latency.
- Response streaming for sub-5-second p95 perceived latency.
- Parallelized synthesis–evaluation execution where safe, removing the sequential bottleneck identified in Section 6.3.
- Blog post on .NET RAG patterns drawn from the LIBRAIN implementation.

The split reflects an evaluation of the marginal value of Phase 3.B for the immediate dissemination goal. Phase 3.A, comprising static demos and portfolio integration, provides the read-only, deterministic showcase a reviewer or recruiter needs to evaluate the system without operating it. Phase 3.B adds an interactive layer whose marginal cost (cloud deployment, caching, streaming) is substantial relative to its marginal evidentiary value over the static demonstrations. Phase 3.B is therefore retained as an explicit deferred-or-optional track rather than treated as a blocking ship gate, and is sequenced after observing whether downstream consumers of the project specifically require interactive demonstration.

7.5 Phase 4: Documentation & Dissemination

- Final results compilation, system-architecture write-up, and update of this document for public release.
- Short technical preprint covering the multi-dimensional judging design and the Cosmos-to-Qdrant pivot as case studies.

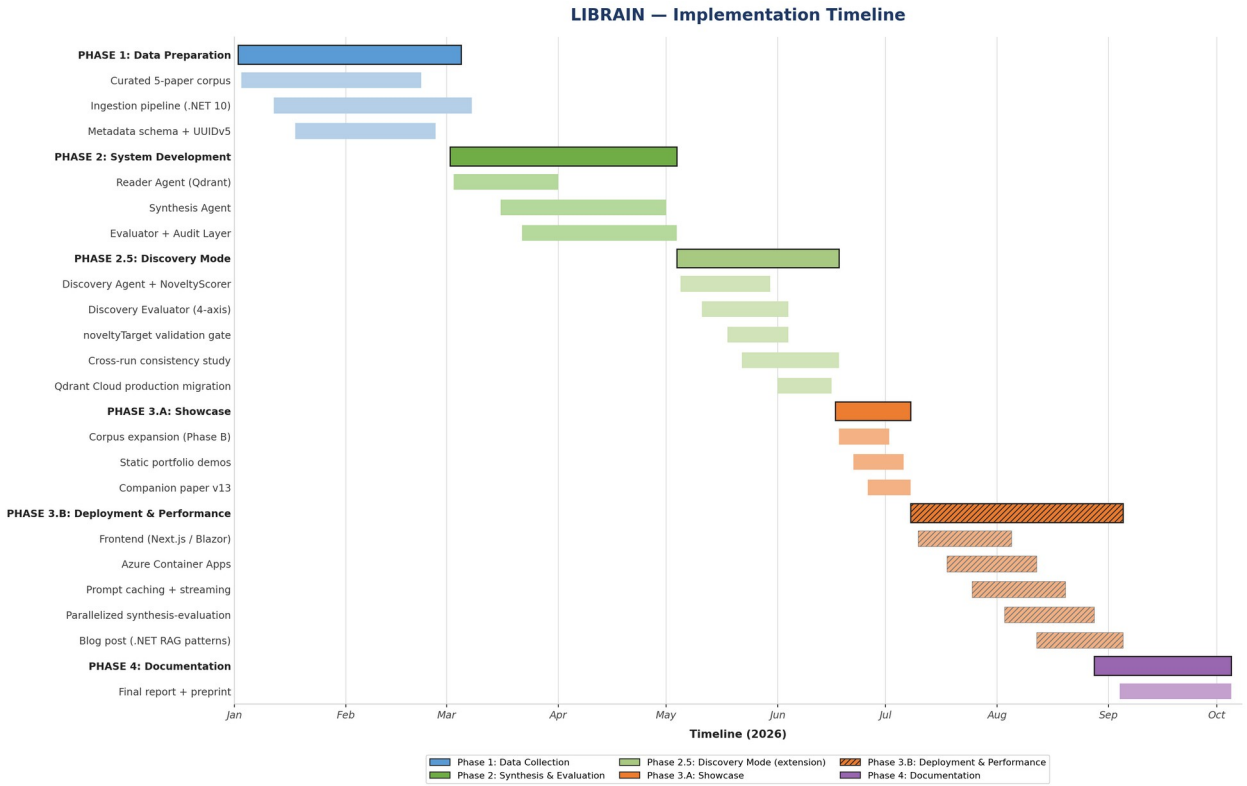

Figure 5. Implementation timeline showing the five phases (Phase 1, 2, 2.5, 3, 4) from data preparation through final dissemination. Phase 2.5 was scoped after Phase 2 shipped and is reflected in the Discovery Agent, NoveltyScorer, cross-run consistency study, and the Qdrant Cloud production migration. Phase 3 is split into Phase 3.A (Showcase), covering corpus expansion and static portfolio demos completed in Phase B (May 2026), and Phase 3.B (Deployment & Performance), covering deferred or optional cloud deployment, streaming, prompt caching, and blog dissemination. The "Cosmos DB migration" bar shown in earlier roadmap drafts has been retired post-Phase 2.5; production storage is now Qdrant Cloud.

8. WIDESPREAD IMPACT

This framework is designed to address specific inefficiencies in current research methodologies. Rather than replacing scientists, LIBRAIN targets tangible bottlenecks in the discovery pipeline.

8.1 Reducing Literature Review Overhead

Researchers currently spend a significant fraction of their time finding and filtering relevant papers. LIBRAIN automates the reading and sorting phase: vector-based retrieval filters out irrelevant material faster than manual triage, allowing researchers to spend their attention on analysis rather than

acquisition. In the prototype, retrieval over a small focused corpus runs well under 200 ms, and even with the full pipeline (synthesis plus evaluation) end-to-end response stays in the tens of seconds.

8.2 Breaking Information Silos (Cross-Domain Discovery)

Scientific fields often become silos in which a researcher in one discipline can miss a relevant solution published in another, simply because the terminology differs. Because the Reader Agent uses semantic embeddings (matching meanings rather than keywords) and because metadata is uniformly attached to every chunk, the system can in principle identify relevant connections across disciplines that strictly keyword-based search engines would miss. Cross-domain stress-testing against a deliberately heterogeneous corpus is a natural extension of this capability and was carried out in Phase B (§5.5 and §6.6): ten topic pairs ran against a 13-paper corpus deliberately spanning drug discovery, climate forecasting, and computational neuroscience, producing cherry-picked Discovery Mode outputs that bridge, for example, retrieval-augmented generation and de novo molecular design (Demo A), weather foundation models and renewable energy planning (Demo B), and drug discovery and climate adaptation (Demo C); these bridges are explicit in the retrieved evidence yet absent from any single retrieved chunk, which is precisely the cross-domain capability §8.2 anticipates.

8.3 Solving the Black-Box Problem in AI Assistance

Most general-purpose AI tools provide answers without exposing the path that produced them, leading to legitimate trust issues in serious research contexts. LIBRAIN's audit-logging architecture ensures that every generated hypothesis carries a citation trail and a per-stage reasoning log via Application Insights. This moves AI assistance from a magic box toward an auditable research tool, adhering to the standards of scientific reproducibility and engineering observability simultaneously.

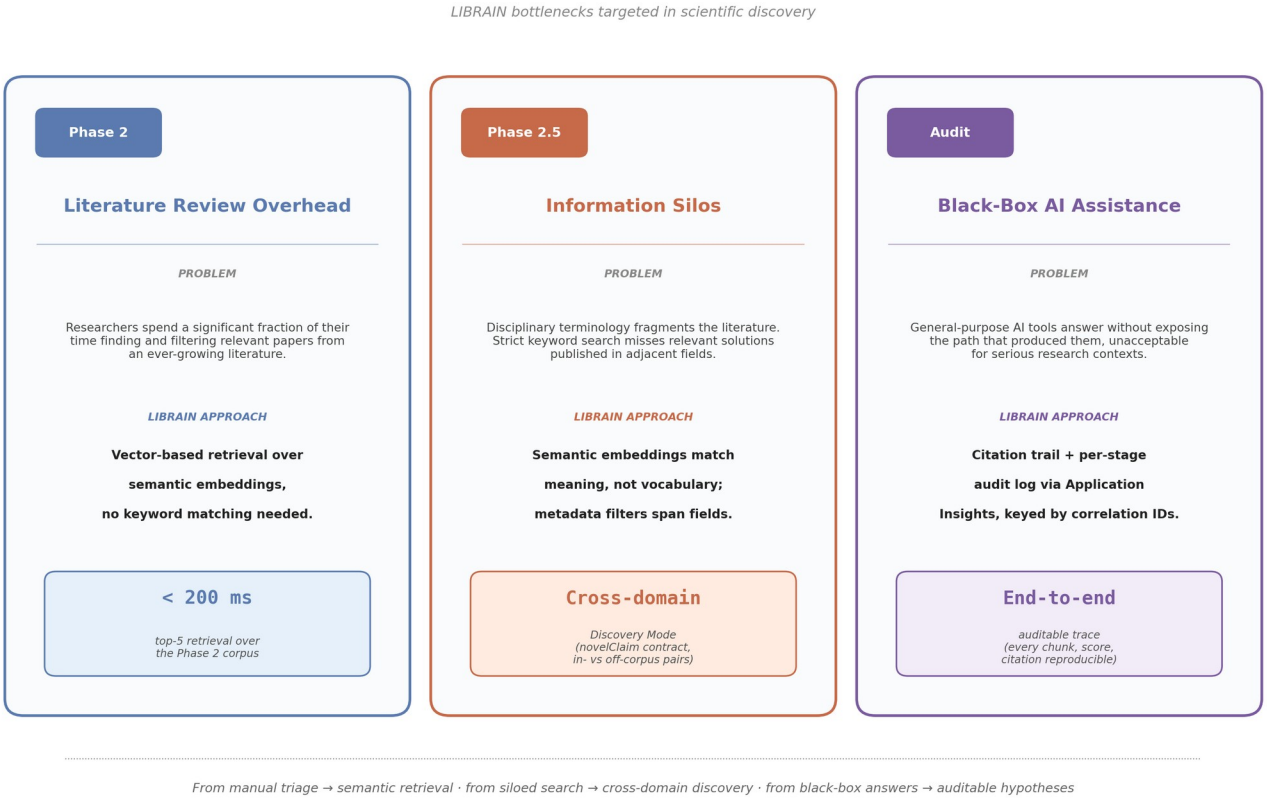

Figure 6. LIBRAIN bottlenecks targeted in scientific discovery. The three impact areas of Section 8, retrieval overhead (Section 8.1), information silos (Section 8.2), and the black-box problem (Section 8.3), are each paired with the LIBRAIN mechanism that addresses them and the concrete measurable outcome observed in the Phase 2 / Phase 2.5 prototype.

9. REFERENCES

[1] M. Gridach, J. Nanavati, K. Z. El Abidine, L. Mendes, and C. Mack, "Agentic AI for Scientific Discovery: A Survey of Progress, Challenges, and Future Directions," arXiv preprint arXiv:2503.08979, Mar. 2025.

[2] A. K. Alkan et al., "A Survey on Hypothesis Generation for Scientific Discovery in the Era of Large Language Models," arXiv preprint arXiv:2504.05496, Apr. 2025.

[3] S. Kulkarni and A. Alotaibi, "Scientific Hypothesis Generation and Validation: Methods, Datasets, and Future Directions," arXiv preprint arXiv:2505.04651, May 2025.

[4] Z. Chen et al., "From AI for Science to Agentic Science: A Survey on Autonomous Scientific Discovery," arXiv preprint arXiv:2508.14111, Aug. 2025.

[5] J. Zhang and L. Wei, "Optimizing Retrieval-Augmented Generation (RAG) for Scientific Literature Mining," Knowledge-Based Systems, vol. 289, pp. 112–125, 2025.

[6] D. A. Boiko, R. MacKnight, and G. Gomes, "Emergent autonomous scientific research capabilities of large language models," Nature, vol. 624, pp. 160–168, 2023.

[7] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative Agents: Interactive Simulacra of Human Behavior," in Proc. of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23), 2023.

[8] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 9459–9474, 2020.

[9] OpenAI, "GPT-4 Technical Report," arXiv preprint arXiv:2303.08774, 2023.

[10] Anthropic, "Claude 4 model family, system documentation," Anthropic technical documentation, 2025–2026.

[11] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," arXiv preprint arXiv:2309.15217, 2023. (Multi-dimensional judging precedent.)

[12] TruEra / TruLens, "TruLens: Evaluation and tracking for LLM applications," open-source documentation, 2023–2025.

[13] Microsoft, ".NET 10 release notes and runtime documentation," Microsoft Learn, 2025–2026.

[14] Qdrant Solutions GmbH, "Qdrant, Open-source vector database, version 1.17," qdrant.tech documentation, 2026.

[15] UglyToad, "PdfPig, read and extract text from PDFs in pure C#," GitHub project documentation, 2025.

[16] Microsoft, "Azure Cosmos DB Vector Search for NoSQL API," Azure documentation, 2025–2026.